

SYSTEM ARCHITECTURE DOCUMENT V2.0

DIY Robot Challenge 2026 — System Architecture

Full technical architecture for an autonomous outdoor obstacle-course robot. ROS 2 Humble on Jetson Orin Nano, multi-level EKF fusion (FAST-LIO2 + dead-wheel encoders + ACEINNA IMU), Nav2 navigation, no GPS dependency.

COMPETITION

October 1, 2026

FEATURE LOCK

August 15, 2026

VERSION

2.0 — May 2026

TEAM

Betsybots

ROS 2 Humble

FAST-LIO2

Nav2

Hesai QT64

RealSense D435i

ACEINNA MTLT335D

Dead-Wheel Encoders

Jetson Orin Nano 8GB

-
- 1 System Overview & Goals
 - 2 Hardware Architecture & Sensors
 - 3 Compute Architecture — Jetson Orin Nano + STM32
 - 4 ROS 2 Software Stack
 - 4.1 Driver Layer
 - 4.2 Localization — Multi-Level EKF (No GPS)
 - 4.3 Perception — PCL Obstacle Classifier
 - 4.4 Navigation — Nav2 (Smac + MPPI)
 - 4.5 Mission Layer — State Machine
 - 5 Multi-Level EKF Fusion Design
 - 6 GPU Acceleration & cuVSLAM
 - 7 Offline Pre-Competition Pipeline
 - 8 Obstacle Handling Strategy
 - 9 Safety & E-Stop Architecture
 - 10 Sensor Calibration Procedures
 - 11 Development Timeline
 - 12 Key Decisions & Rejected Alternatives
 - 13 Risks & Mitigations

1.1 Competition Context

The CAT Robotics Culture Club DIY Robot Challenge 2026 is an autonomous outdoor obstacle-course race. Teams build custom robots competing on a **speed course** (3 laps) and an **obstacle course** (2 laps). Scoring is time-based with cumulative penalties for skipped obstacles.

PARAMETER	VALUE	NOTES
Max dimensions	16" W × 24" L × 16" H	Hard limit
Max weight	25 lbs	Including all electronics
Obstacle course	2 consecutive laps	Timed, best heat used
Skip penalty	15s × n per skip	Cumulative — skipping 5 = 15×15 = 225s total
Min track width	20 inches	Robot is 16" — only 2" clearance each side
Heat duration	10 minutes	Multiple runs per heat
Communication	None during run (Rule 2.4)	E-Stop exempt

1.2 Design Goals

MVP — Reliable 2-Lap Completion

Complete both laps of the obstacle course within 10 minutes using correct behaviors (push-through vs navigate-around) for each obstacle type. Zero GPS dependency — localization entirely from LiDAR + IMU + encoders against pre-built map.

Stretch — Sub-4-Minute Run

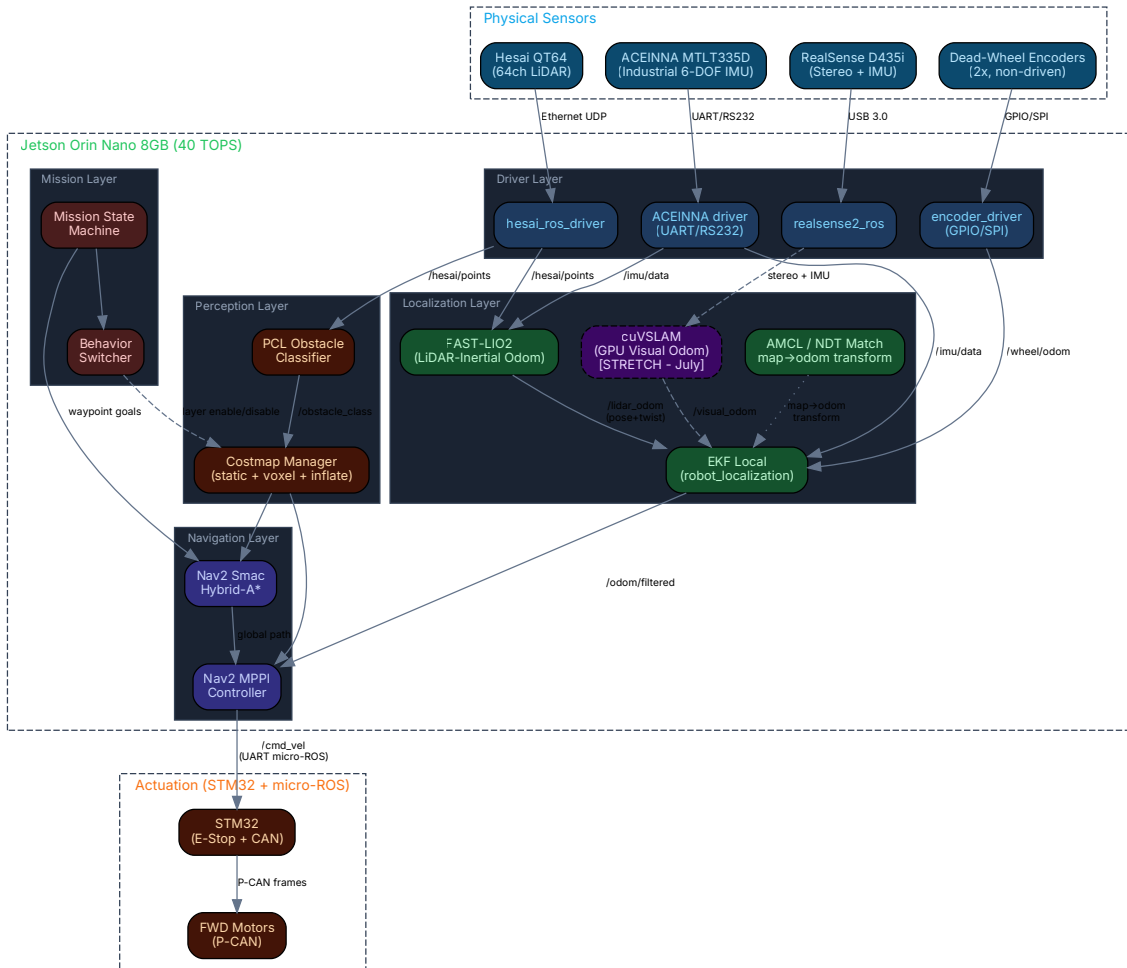
Zero obstacle skips, consistent 2-lap completion under 4 minutes with sub-5cm localization accuracy at critical waypoints. cuVSLAM provides visual-inertial redundancy in case of LiDAR degradation.

1.3 Key Changes from v1.0 Architecture

COMPONENT	V1.0 (PREVIOUS)	V2.0 (CURRENT)	REASON
GPS	RTK GNSS (\$2500 BX992)	REJECTED — No GPS	\$2500 + base station + correction service complexity. LiDAR localization sufficient.
Compute	Jetson Nano (4GB)	Jetson Orin Nano (8GB, 40 TOPS)	10× GPU, 2× RAM, same form factor. Enables cuVSLAM.
IMU	Generic 6-DOF	ACEINNA MTLT335D (industrial)	1.3°/hr bias stability, CAN/RS232. Dramatically better drift.
Wheel Odom	Removed (slip issues)	Dead-wheel encoders (non-driven)	Non-driven wheels don't slip. Fills gaps between LiDAR scans.
Global Loc			

COMPONENT	V1.0 (PREVIOUS)	V2.0 (CURRENT)	REASON
	RTK EKF2 + navsat_transform	AMCL/NDT match against pre-built map	No GPS means map-matching provides map→odom.
GPU Odom	None	cuVSLAM (stretch, July)	Free GPU cycles on Orin — visual odom as redundancy.
Feature Lock	Sep 15	Aug 15 (45-day buffer)	More practice time before competition.

FIGURE 1 — FULL SYSTEM ARCHITECTURE



2.1 Sensor Inventory

COMPONENT	MODEL	INTERFACE	ROLE	NOTES
3D LiDAR	Hesai QT64	Ethernet (UDP)	FAST-LIO2 odom, PCL classifier, costmap	64ch, $\pm 52^\circ$ VFOV, ~ 20 Hz
IMU	ACEINNA MTLT335D	UART / RS232	FAST-LIO2 fusion, EKF prediction	Industrial 6-DOF, $1.3^\circ/\text{hr}$ gyro bias, CAN/RS232
Camera	RealSense D435i	USB 3.0	cuVSLAM (stretch), AprilTag calib, visual start	Stereo + built-in IMU. On-sensor depth ASIC.
Encoders	Dead-wheel ($\times 2$)	GPIO / SPI	Twist input to EKF, fills LiDAR gaps	Non-driven — no slip. Mounted on passive wheels.
Compute	Jetson Orin Nano 8GB	—	All ROS 2 stack	40 TOPS GPU, 6 ARM cores, 8GB RAM

COMPONENT	MODEL	INTERFACE	ROLE	NOTES
Microcontroller	STM32	UART (micro-ROS)	Motor control, E-Stop	Hard real-time, CAN bus master
Motors	FWD Controllers	P-CAN	Drive + steering	Via STM32 CAN bridge
E-Stop (Wired)	Course-provided	GPIO → STM32	Safety stop (Rule 1.2.2)	Hardwired, bypasses Jetson
E-Stop (Wireless)	Team RF unit	RF → STM32	300ft range (Rule 1.2.4)	Fail-safe: stop if signal lost >1s

WHY DEAD-WHEEL ENCODERS INSTEAD OF MOTOR ENCODERS?

FWD motor encoders measure wheel rotation — but on gravel, inclines, and turns the driven wheels slip, producing false odometry that corrupts the EKF. Dead-wheel encoders are mounted on passive, non-driven wheels that maintain ground contact without slip. They provide reliable twist (dx, dy, dθ) at 50-100 Hz that fills the gap between 10-20 Hz LiDAR scans.

3.1 Responsibilities Split

STM32 — Real-Time Layer

- CAN bus: /cmd_vel → motor frames
- E-Stop: wired + wireless → cmd=0 within 1s
- Watchdog: wireless signal lost >800ms → auto-stop
- micro-ROS bridge over UART to Jetson

Jetson Orin Nano — ROS 2 Stack

- All sensor drivers (LiDAR, IMU, camera, encoders)
- FAST-LIO2 LiDAR-inertial odometry (CPU)
- robot_localization EKF (CPU)
- Nav2 planner + controller (CPU)
- cuVSLAM visual odometry (GPU — stretch)
- Mission state machine + behavior switcher

3.2 Compute Load Estimate

PROCESS	CPU %	GPU %	RAM	NOTES
FAST-LIO2	~20%	0%	~400MB	Primary odom
Nav2 (planner + MPPI)	~18%	0%	~300MB	Main CPU consumer
PCL_classifier	~8%	0%	~150MB	Per-scan processing
hesai_ros_driver	~4%	0%	~100MB	UDP parse only
realsense2_ros	~5%	0%	~200MB	Depth on-sensor ASIC
cuVSLAM (stretch)	~2%	~25%	~300MB	Runs entirely on GPU
robot_localization	~2%	0%	~50MB	Lightweight EKF
Total	~60%	~25%	~1.5GB	Plenty of headroom

ORIN NANO ADVANTAGE

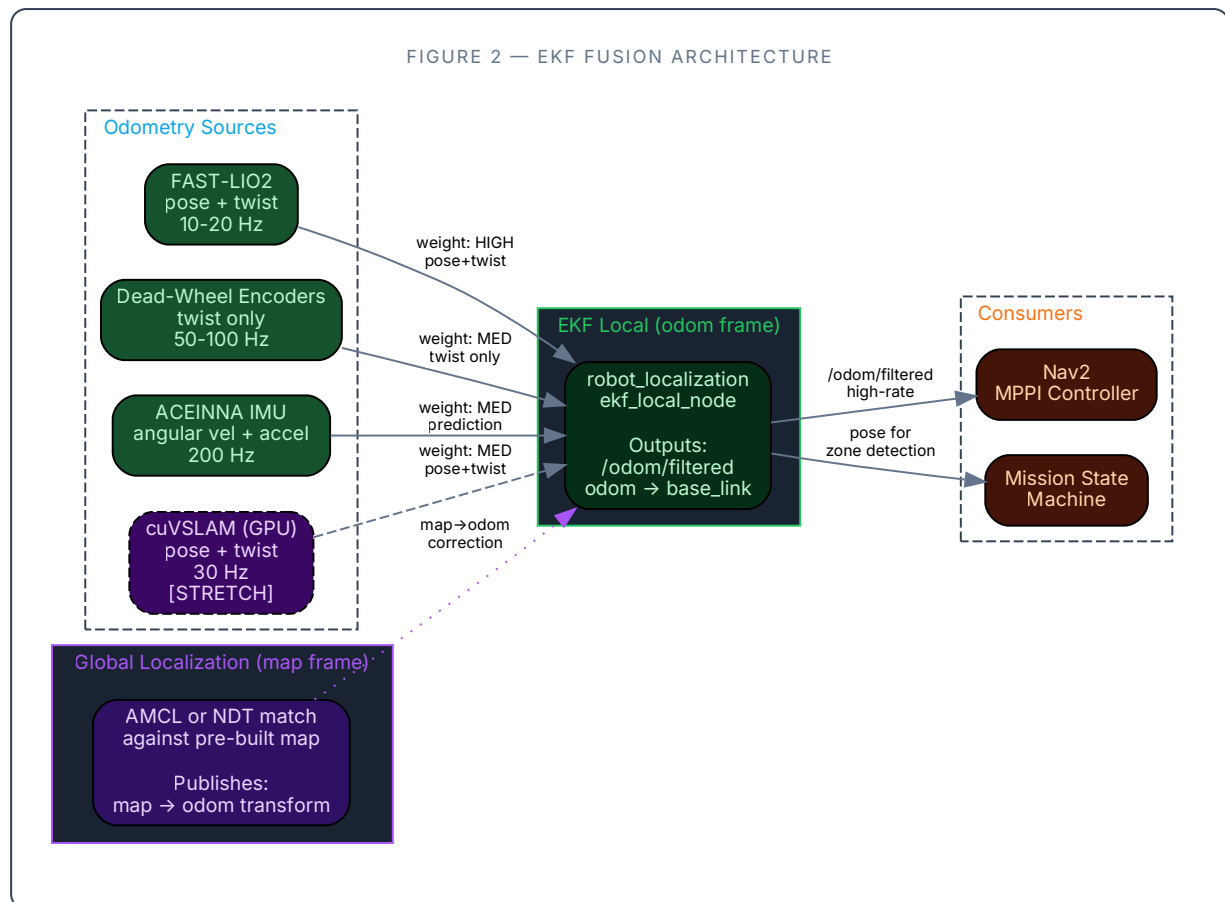
With 6 ARM cores and 8GB RAM, the Orin Nano has ~40% CPU headroom vs the old Jetson Nano's ~15%. The 40 TOPS GPU is almost entirely idle without cuVSLAM — making it a free resource for stretch goals.

4.1 Driver Layer

NODE	INPUT	OUTPUT TOPIC	RATE
<code>hesai_ros_driver</code>	Ethernet UDP	<code>/hesai/points</code> (PointCloud2)	~20 Hz
<code>aceinna_driver</code>	UART/RS232	<code>/imu/data</code> (Imu)	200 Hz
<code>realsense2_ros</code>	USB 3.0	<code>/camera/color/image_raw</code> , <code>/camera/depth/...</code>	30 Hz
<code>encoder_driver</code>	GPIO/SPI	<code>/wheel/odom</code> (Odometry, twist only)	50-100 Hz
<code>micro_ros_agent</code>	UART (STM32)	relay of <code>/cmd_vel</code>	on demand
<code>joy_node</code>	USB gamepad	<code>/cmd_vel</code> (mapping only)	50 Hz

4.2 Localization — Multi-Level EKF (No GPS)

With RTK GPS rejected, localization relies entirely on **FAST-LIO2 + dead-wheel encoders + ACEINNA IMU** fused in a local EKF, with AMCL/NDT providing the global map→odom correction.



FAST-LIO2 — PRIMARY ODOMETRY

PROPERTY	VALUE
Algorithm	Iterative Extended Kalman Filter (iEKF) on point-to-plane
Inputs	<code>/hesai/points</code> + <code>/imu/data</code>
Output	<code>/lidar_odom</code> (Odometry) @ 10-20 Hz
Drift (calibrated)	0.1-0.3% of distance traveled
Tunnel behavior	Rich wall features — no degradation

EKF LOCAL — ROBOT_LOCALIZATION

Inputs:

- `/lidar_odom` (FAST-LIO2, pose+twist, 10-20Hz) — weight: HIGH
- `/wheel/odom` (encoders, twist only, 50-100Hz) — weight: MEDIUM
- `/imu/data` (ACEINNA, angular vel, 200Hz) — weight: MEDIUM (prediction)
- `/visual_odom` (cuVSLAM, pose+twist, 30Hz) — weight: MEDIUM [STRETCH]

Output: `/odom/filtered` (high-rate fused odometry)

Frame: `odom` → `base_link`

Purpose: Feeds Nav2 MPPI controller — low latency, high rate

GLOBAL LOCALIZATION — AMCL / NDT MATCH

Method: Match live LiDAR scans against pre-built map

Output: `map` → `odom` transform (corrects accumulated drift)

Rate: ~1-5 Hz (matching frequency)

Purpose: Provides global consistency without GPS

Note: No `navsat_transform` needed — purely map-based

4.3 Perception — PCL Obstacle Classifier

Pipeline (per LiDAR scan, ~20Hz):

1. Ground removal → RANSAC plane fit, remove points within 5cm
2. Clustering → Euclidean extraction (tolerance: 0.15m)
3. Map differencing → Compare vs `prior_map.pcd` (new obstacles only)
4. Classification → Per-cluster: CHIMES / MOVABLE / TUNNEL / WALL

Output: `/obstacle_class` (`zone_id`, `classification`, `confidence`)

4.4 Navigation — Nav2

● Global: Smac Hybrid-A*

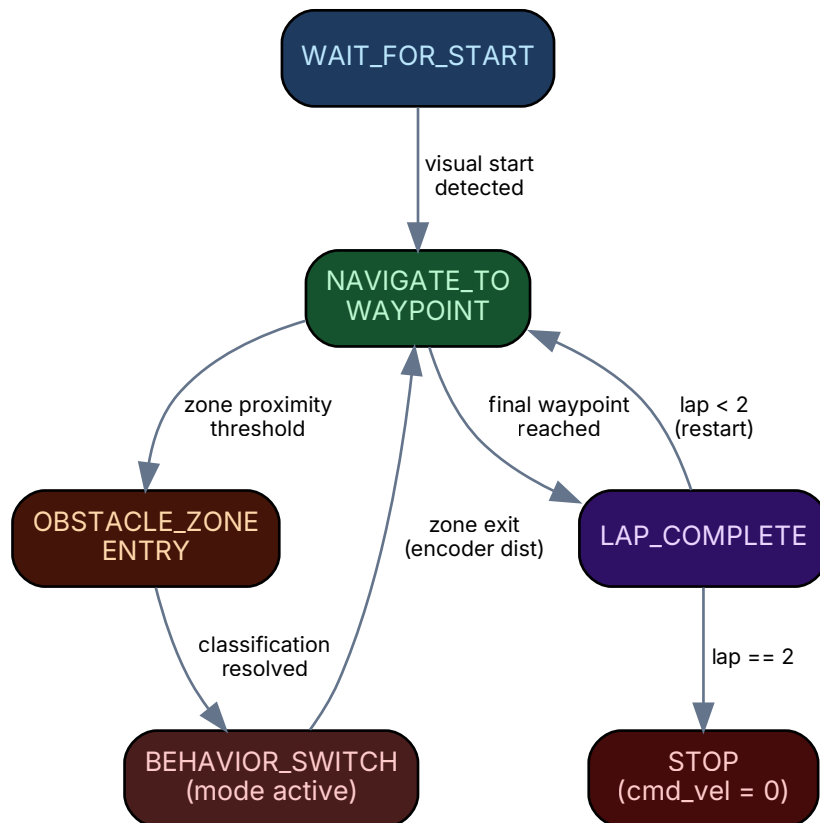
● Local: MPPI Controller

Non-holonomic path planning respecting turn radius. Pre-computed from centerline waypoints. Re-invoked only on WALL classification.

Samples thousands of trajectory rollouts. Smooth velocity profiles on gravel/ramps. Better than DWB for dynamic obstacles.

4.5 Mission Layer — State Machine

FIGURE 3 — MISSION STATE MACHINE



BEHAVIOR MODE	TRIGGER	ACTIONS	EXIT
PUSH_THROUGH	TUNNEL, CHIMES	Disable voxel costmap, hold heading, 0.2 m/s	Encoder distance past zone
NAVIGATE_AROUND	MOVABLE object	Keep costmap, re-plan path around	Past obstacle + 0.5m
BLIND_DRIVE	CHIMES (fallback)	All obstacle layers off, straight, 0.15 m/s	Distance > zone_width
NORMAL_NAV	Default	Full Nav2 active	Zone entry detected

This is the core localization architecture. With no GPS, all global consistency comes from map matching. The EKF fuses multiple odometry sources for high-rate, low-latency local pose estimates.

5.1 Source Characteristics

SOURCE	PROVIDES	RATE	STRENGTH	WEAKNESS
FAST-LIO2	Full 6DOF pose + twist	10-20 Hz	Best absolute accuracy (0.1-0.3% drift)	Degrades in featureless areas, lower rate
Dead-wheel encoders	2D twist (dx, dy, dθ)	50-100 Hz	High rate, fills LiDAR gaps, no slip	2D only, no vertical, accumulates drift
ACEINNA IMU	Angular vel + linear accel	200 Hz	Highest rate, prediction between updates	Double-integration drift (mitigated by 1.3°/hr bias)
cuVSLAM (stretch)	Full 6DOF pose + twist	30 Hz	GPU-only (0% CPU), independent modality	Fails in darkness/textured-poor areas

5.2 EKF Configuration

```
# ekf_local.yaml - robot_localization config
ekf_local_node:
  ros__parameters:
    frequency: 50.0
    two_d_mode: false
    publish_tf: true
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom

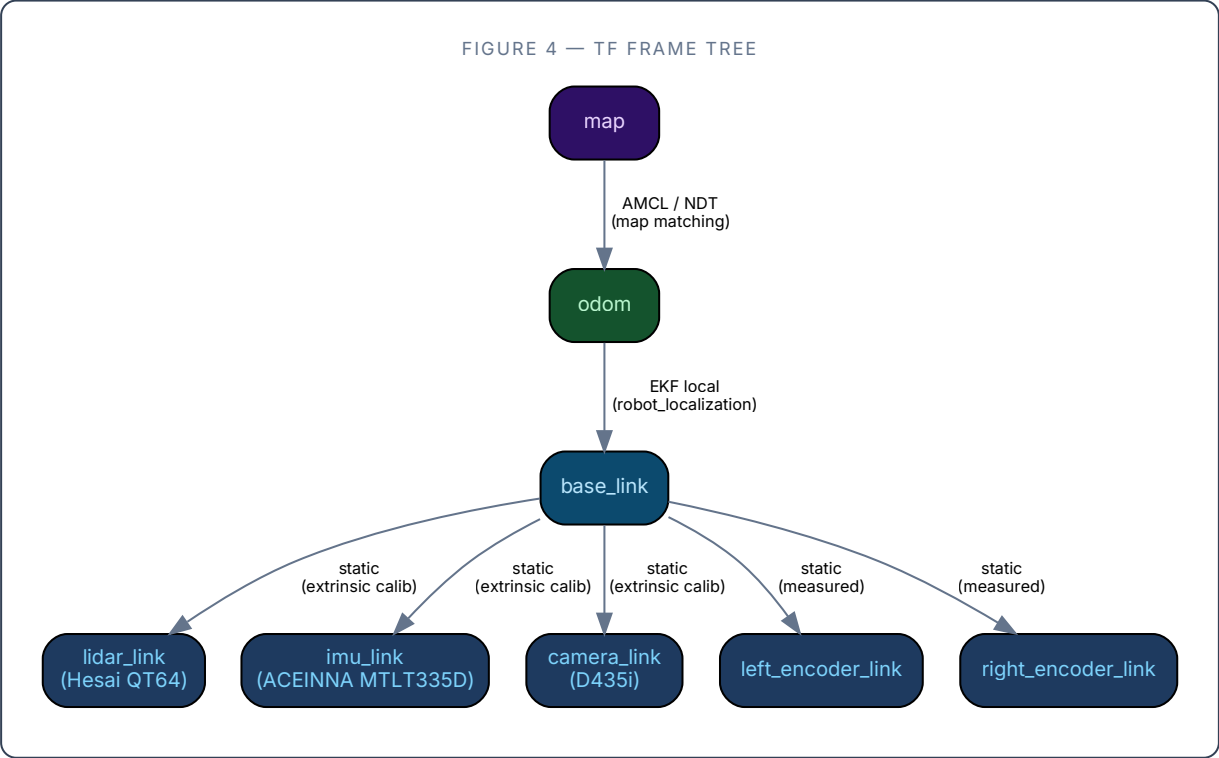
    # FAST-LIO2: pose + twist (highest weight)
    odom0: /lidar_odom
    odom0_config: [true, true, true,      # x, y, z
                  true, true, true,      # roll, pitch, yaw
                  true, true, true,      # vx, vy, vz
                  true, true, true,      # vroll, vpitch, vyaw
                  false, false, false]   # ax, ay, az

    # Dead-wheel encoders: twist only
    odom1: /wheel/odom
    odom1_config: [false, false, false,
                  false, false, false,
                  true, true, false,     # vx, vy (2D twist)
                  false, false, true,    # vyaw
                  false, false, false]

    # ACEINNA IMU: angular velocity + linear acceleration
    imu0: /imu/data
    imu0_config: [false, false, false,
                  false, false, false,
                  false, false, false,
                  true, true, true,      # vroll, vpitch, vyaw
                  true, true, true]      # ax, ay, az
```

```
# cuVSLAM (stretch): pose + twist
# odom2: /visual_odom
# odom2_config: [true, true, true, true, true, true,
#               true, true, true, true, true, true,
#               false, false, false]
```

5.3 TF Tree



CRITICAL — NO NAVSAT_TRANSFORM

With GPS removed, there is NO EKF2/navsat_transform. The map→odom transform comes from AMCL or NDT matching the live scan against the pre-built map. This is simpler but requires a high-quality prior map. Map building is the #1 priority in June.

5.4 Accuracy Expectations (No GPS)

SCENARIO	EXPECTED ACCURACY	NOTES
FAST-LIO2 alone, calibrated	5–15cm per full lap	Outdoor, mixed terrain
+ Encoders + IMU in EKF	3–8cm per lap	Encoder twist fills scan gaps
+ AMCL map matching (open area)	<5cm absolute	Continuous drift correction
+ cuVSLAM redundancy (stretch)	<3cm absolute	Independent visual confirmation
Inside tunnel (no GPS anyway)	~5–10cm drift over length	Rich wall features for LiDAR

NARROW PASSAGE REQUIREMENT

Track width 20" – robot 16" = 4" total clearance (2" per side = 5cm). Need lateral accuracy <4cm at passage entry. AMCL correction immediately before narrow passages provides this. Failsafe: slow to 0.1 m/s and hold heading through passage.

The Jetson Orin Nano's 40 TOPS GPU is largely idle in the base configuration. This section documents what can (and cannot) be offloaded to GPU.

6.1 CUDA Offloading Reality Check

COMPONENT	CUDA ACCELERATED?	REALITY
Hesai LiDAR driver	No	Just UDP packet parsing — trivial CPU load (~4%)
RealSense driver	No	Depth computed on-sensor ASIC, not host CPU
Isaac ROS image_proc	Yes	GPU rectification/resize of camera images
Isaac ROS depth_image_proc	Yes	GPU depth→pointcloud conversion
cuVSLAM	Yes	Full visual SLAM on GPU — zero CPU cost
Nvblox	Yes	3D costmap on GPU (replaces CPU costmap2D)
FAST-LIO2	No	CPU-only iEKF — no CUDA path exists
Nav2 planner/controller	No	CPU-only — no CUDA path exists
PCL/NDT	Partial	ndt_omp uses OpenMP (CPU threads), not CUDA

BOTTOM LINE

The raw sensor drivers are NOT CPU-heavy — there's nothing meaningful to offload. The real GPU value comes from cuVSLAM (visual odometry) and potentially Nvblox (3D costmap). FAST-LIO2 and Nav2 remain CPU-bound regardless.

6.2 cuVSLAM Integration Plan (July Stretch Goal)

STEP	ACTION	TIME ESTIMATE
1	Install Isaac ROS cuVSLAM package (apt)	30 min
2	Configure for D435i stereo + built-in IMU	1 hour
3	Verify <code>/visual_odom</code> output in RViz	1 hour
4	Add as <code>odom2</code> in <code>ekf_local.yaml</code>	30 min
5	Test: disable FAST-LIO, run on cuVSLAM alone	1 hour
6	Tune relative weights in EKF	2 hours

DO NOT TOUCH UNTIL JULY

cuVSLAM is a stretch goal. The base stack (FAST-LIO + encoders + IMU) must work perfectly first. Adding cuVSLAM prematurely introduces debugging complexity. Only add it when the base EKF is producing clean, consistent laps.

6.3 Nvblox — 3D Costmap (August Stretch)

If cuVSLAM works well in July, consider Nvblox in Aug 1-15. Nvblox builds a GPU-accelerated 3D TSDF/ESDF costmap from depth images. Benefits: better obstacle representation than 2D voxel layer, handles overhanging obstacles. Risk: adds complexity near feature lock. *Only pursue if specific obstacle types require 3D reasoning.*

7.1 Phase 1 — Prior Map Building (June)

STEP	ACTION	OUTPUT
1	Joystick drive — 2 full laps at 0.3 m/s (clean track, no obstacles)	rosvbag
2	Record: <code>/hesai/points /imu/data /wheel/odom /camera/... /tf /tf_static</code>	~2GB bag
3	Offline LIO-SAM processing (loop closure on lap 2)	<code>prior_map.pcd</code>
4	octomap_server → 2D occupancy grid	<code>static_map.yaml</code>
5	Voronoi skeletonization → centerline waypoints	<code>waypoints.yaml</code>
6	Manual zone boundary annotation	<code>zone_template.yaml</code>

7.2 Phase 2 — Day-Before Classification

STEP	ACTION	OUTPUT
1	Slow scan lap with obstacles present (0.2 m/s)	day-of rosvbag
2	Map diff: subtract <code>prior_map.pcd</code> (threshold 0.1m)	new obstacle points
3	Zone classifier: cluster + classify per zone	behavior per zone
4	Generate <code>zone_config.yaml</code> , verify in RViz	final config
5	Deploy to robot, verify full stack startup	ready for race

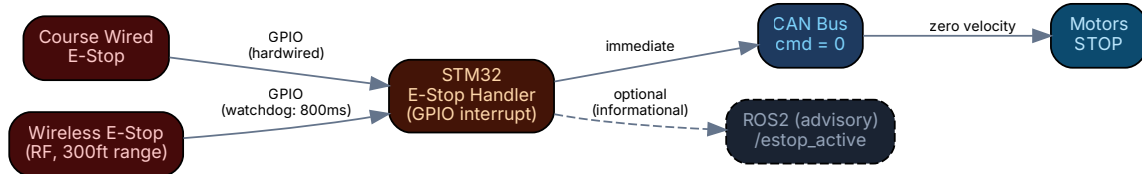
7.3 AprilTag EKF Tuning (Pre-Competition Test Track)

8-10 AprilTags at surveyed positions in your own test track provide 6DOF ground truth. Run 10+ laps → compute Innovation-based Adaptive EKF Q/R matrices. NOT placed on competition course — purely a calibration tool.

Obstacle Handling Strategy

OBSTACLE	DETECTION	CHALLENGE	BEHAVIOR
Tunnel	Zone boundary + bounded walls	No sky (irrelevant — no GPS anyway). Rich LiDAR features.	PUSH_THROUGH
Ramp	Zone + IMU pitch change	Surface change. FAST-LIO2 handles natively.	NORMAL_NAV (slow)
Chimes	PCL: thin vertical clusters	Ground-touching, can't scan below.	PUSH_THROUGH / BLIND_DRIVE
Movable	PCL: isolated dense + map diff	Nudging corrupts lap-2 map.	NAVIGATE_AROUND
Narrow passage	Zone + walls close on sides	2" clearance. Need <4cm lateral accuracy.	PUSH_THROUGH (slow)
Gravel	Zone + IMU vibration	Dead-wheel encoders handle surface change.	NORMAL_NAV (slower)

FIGURE 5 — E-STOP DATA FLOW

**CRITICAL DESIGN PRINCIPLE**

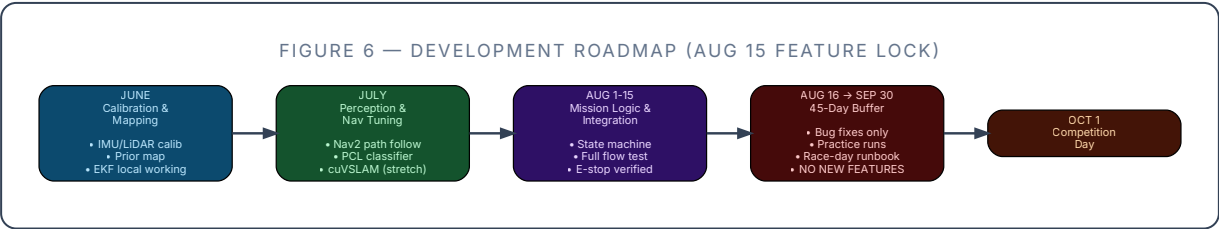
The E-Stop path NEVER passes through ROS 2 or the Jetson. If the Jetson crashes, the E-Stop still works. STM32 handles safety independently via hardware interrupts.

REQUIREMENT	RULE	IMPLEMENTATION
Stop within 1 second	1.2.7.2	STM32 GPIO interrupt → CAN cmd=0 immediately
Wireless signal loss → stop	1.2.7.1	800ms watchdog timer on STM32
Course wired E-Stop	1.2.2	Hardwired to STM32, bypasses Jetson
Team wireless, 300ft	1.2.4	RF receiver → STM32 GPIO

#	PROCEDURE	TOOL	OUTPUT	PRIORITY
1	IMU intrinsic calibration (2-3hr stationary)	imu_utils	Noise density, bias instability → FAST-LIO2 config	Week 1
2	IMU-LiDAR extrinsic (figure-8 motion)	lidar_imu_calib	6DOF transform → URDF + FAST-LIO2	Week 1
3	Camera-LiDAR extrinsic	cam_lidar_calib	Transform → URDF	Week 2
4	Encoder wheel diameter + baseline	Physical measurement	Tick-to-meters conversion	Week 1
5	IMU-LiDAR time sync verification	Visual check (smeared scans)	Timestamp offset correction	Week 2
6	EKF Q/R tuning via AprilTag ground truth	Innovation covariance method	competition.yaml	Late June

CALIBRATION ORDER MATTERS

Each step depends on the previous. IMU intrinsics affect FAST-LIO2 accuracy, which affects extrinsic calibration quality. Do NOT skip to software development until steps 1-4 are complete.



PHASE	DATES	DELIVERABLES	GATE
Calibration & Mapping	June 1–30	All calibrations done. Prior map built. EKF local producing clean odom. TF tree verified.	Clean 1-lap odom in RViz
Perception & Nav	July 1–31	Nav2 path following. PCL classifier working. cuVSLAM added (stretch). E-stop tested.	3 clean autonomous laps
Mission & Integration	Aug 1–15	State machine. Behavior switcher. Full startup→run→stop flow. Obstacle handling end-to-end.	Full 2-lap obstacle run
FEATURE LOCK	Aug 15	No new features after this date. Only bug fixes and practice runs for 45 days.	
Bug Fixes & Practice	Aug 16 – Sep 30	Multiple full-course dry runs. Race-day runbook. Backup procedures. Hardware spares.	5 consecutive clean runs
Competition	Oct 1	Day-before scan + classify. Race morning: stack verify. Trust the system.	—

✗ REJECTED: RTK GPS (BX992 — \$2,500)

Rationale: \$2,500 receiver + \$1,000+/yr correction service + base station setup + antenna integration. LiDAR-based localization (FAST-LIO2 + AMCL) achieves <5cm accuracy without any of this infrastructure. GPS degrades near buildings/trees (common on outdoor courses). The tunnel obstacle makes GPS useless for a significant portion of the track anyway.

What replaces it: AMCL/NDT matching against pre-built map provides map→odom transform. Dead-wheel encoders provide high-rate twist. Combined with FAST-LIO2's excellent accuracy, no GPS needed.

✓ ACCEPTED: DEAD-WHEEL ENCODERS

Rationale: Non-driven wheels don't slip — unlike motor encoders on FWD. Provides reliable 50-100 Hz twist that fills the 50-100ms gaps between FAST-LIO2 LiDAR updates. Critical for smooth MPPI controller inputs. Low cost (~\$50-100 for two rotary encoders + mounting).

✓ ACCEPTED: ACEINNA MTLT335D INDUSTRIAL IMU

Rationale: 1.3°/hr gyro bias stability (vs >10°/hr for consumer IMUs). CAN/RS232 interface (no USB latency jitter). Temperature-compensated. The single largest factor in FAST-LIO2 accuracy is IMU quality — this is worth the cost.

✓ ACCEPTED: CUVSLAM AS STRETCH GOAL (JULY)

Rationale: Runs entirely on Orin Nano GPU (0% CPU impact). Provides independent visual-inertial odometry from D435i stereo. If FAST-LIO2 degrades (featureless area, dust), cuVSLAM maintains localization. 2-3 hour integration once base stack works.

RISK	LIKELIHOOD	SEVERITY	MITIGATION
AMCL map match fails (featureless area)	Low	High	EKF local continues with FAST-LIO2+encoders. Drift is only ~5-15cm/lap without global correction. Acceptable for most obstacles.
Dead-wheel encoder mounting loses contact	Low	Medium	Spring-loaded mount. EKF gracefully degrades — FAST-LIO2 still provides primary odom.
Chimes classified as wall (stops robot)	Medium	High	BLIND_DRIVE_MODE as guaranteed fallback — bypasses all perception.
cuVSLAM destabilizes EKF	Low	Medium	Add in July only. Test extensively. Keep as odom2 with lower weight. Easy to disable if problematic.
Narrow passage lateral drift >4cm	Medium	High	Force AMCL correction before passage entry. Slow to 0.1 m/s. Hold centerline heading.
Prior map quality poor	Medium	High	Build map early (June). Multiple re-recordings. Verify loop closure quality. This is the foundation — invest time here.
TF tree errors found late	Low	High	Build and verify URDF in Week 1. Run tf2_echo on all pairs before any odom work.

Package Reference

PACKAGE	PURPOSE
FAST-LIO	LiDAR-IMU odometry (primary)
robot_localization	EKF fusion (local frame)
nav2_bringup	Navigation stack
nav2_smac_planner	Hybrid-A* global planner
nav2_mppi_controller	MPPI local controller
LIO-SAM	Offline SLAM for prior map
isaac_ros_visual_slam	cuVSLAM (stretch)
pcl_ros	PCL bindings
imu_utils	IMU Allan variance calibration
lidar_imu_calib	LiDAR-IMU extrinsic
micro_ros_agent	STM32 bridge

